

AD23B31 - IMAGE PROCESSING AND COMPUTER VISION

**CONSTRUCTION SITE SAFETY MONITORING SYSTEM
WITH REAL-TIME DANGER ZONE VIOLATION DETECTION**

A MINI PROJECT REPORT

Submitted by

SURYA NIRANJAN S

2116230701354



**RAJALAKSHMI
ENGINEERING
COLLEGE**

March, 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**CONSTRUCTION SITE SAFETY MONITORING SYSTEM WITH REAL-TIME DANGER ZONE VIOLATION DETECTION**" is the bonafide work of **211623070354** who carried out the work under my supervision.

Certified further that to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

The project work has been carried out under my supervision and to my satisfaction. This work is original and has not been submitted elsewhere for the award of any degree or diploma.

SIGNATURE

Dr.S.Madhusudhanan

Associate Professor

Department of Computer Science Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voice Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Construction site safety is a critical global concern, with workers routinely exposed to hazardous environments involving heavy machinery, deep excavations, and restricted zones. Manual monitoring of such environments is impractical and prone to error. This project presents SITEGUARD, an automated real-time construction site safety monitoring system designed to detect and alert when personnel enter predefined danger zones in video footage.

The system employs MobileNet-SSD for person detection, operating with a multi-scale inference pipeline that enhances detection of small or partially occluded workers. A configurable polygon danger zone is defined per scene, and a point-in-polygon test using OpenCV's `cv2.pointPolygonTest` determines whether any detected person has entered the restricted area. Upon violation detection, the system triggers timestamped alerts and captures snapshot images for documentation.

The system is deployed as a Flask-based web application named SITEGUARD, featuring browser-based login authentication, video file upload, live MJPEG stream display, real-time statistics, violation snapshot gallery, and event log viewing. The solution operates efficiently on standard CPU hardware, making it suitable for deployment in practical construction site monitoring scenarios.

Keywords: Construction Site Safety, Person Detection, MobileNet-SSD, Danger Zone, Polygon Zone Detection, Flask, Computer Vision, MJPEG Streaming, Safety Monitoring.

TABLE OF CONTENTS

S.No	Contents	Page No.
1.	Introduction	
2.	Problem Statement	
3.	Objectives	
4.	Literature Survey	
5.	System Requirements	
6.	Methodology	
7.	System Architecture	
8.	System Flow Diagram	
9.	Technology Stack	
10.	Working of the System	
11.	Implementation Details	
12.	Results and Discussion	
13.	Advantages	
14.	Limitations	
15.	Future Enhancements	
16.	Conclusion	
17.	References	

CHAPTER 1: INTRODUCTION

1.1 Background

Construction sites represent one of the most hazardous working environments globally. Workers are routinely exposed to risks including proximity to heavy earthmoving machinery, falls into deep excavations, and inadvertent entry into restricted zones where active digging, drilling, or lifting operations are in progress. According to occupational safety statistics, construction sites account for a disproportionate share of workplace fatalities, particularly in developing nations with rapidly expanding infrastructure projects.

Traditional approaches to construction site safety rely on physical barriers, manual supervision by safety officers, and periodic audits. These methods are inherently limited by human attention span, shift coverage gaps, and the inability to monitor multiple zones simultaneously. There exists a pressing need for an automated, intelligent monitoring solution capable of continuously surveilling restricted areas and issuing immediate alerts upon unauthorized entry.

1.2 Motivation

Advances in computer vision have made real-time person detection and spatial zone monitoring feasible on standard CPU hardware. Classical object detectors such as MobileNet-SSD, originally designed for mobile and edge deployment, offer an effective balance of detection speed and accuracy without requiring GPU infrastructure. The availability of the OpenCV deep neural network module further simplifies deployment on diverse hardware configurations.

Simultaneously, the maturation of Flask as a lightweight Python web framework enables rapid development of browser-based monitoring dashboards that can serve live video streams, display real-time statistics, and archive violation evidence. The combination of these technologies motivates the development of SITEGUARD — a practical, deployable construction site safety monitoring system.

1.3 Project Overview

SITEGUARD is an automated construction site safety monitoring system developed as a Flask-based web application. The system follows a two-stage pipeline: MobileNet-SSD first detects persons in each video frame, after which a configurable polygon danger zone is used to determine whether any detected person has entered the restricted area. Violations trigger timestamped log entries and snapshot captures. The web interface provides secure login, video upload, live MJPEG feed display, real-time statistics, and violation evidence browsing.

1.4 Scope of the Project

The scope of this project includes:

- Real-time person detection using MobileNet-SSD with multi-scale inference
- Configurable polygon danger zone definition via interactive calibration tool

- Point-in-polygon zone membership testing using OpenCV
- Violation alerting with timestamped log entries and snapshot captures
- Flask web application with login authentication and video upload
- Live MJPEG stream display with real-time statistics dashboard
- Violation snapshot gallery and event log browser

CHAPTER 2: PROBLEM STATEMENT

Construction site accidents involving unauthorized entry into active danger zones — areas with operating heavy machinery, deep excavations, or active material drops — remain a persistent occupational safety challenge. Manual supervision is insufficient given the scale of modern construction projects, where dozens of workers operate simultaneously across multiple zones.

Existing automated systems are often computationally expensive, requiring GPU infrastructure that is impractical for deployment on standard site monitoring hardware. Additionally, many systems lack the flexibility to define custom danger zones that match the specific geometry of a given construction scene.

There is a pressing need for an intelligent and affordable construction site safety monitoring system that:

- Detects workers in real time from video footage on standard CPU hardware
- Supports configurable polygon danger zones adaptable to any scene geometry
- Provides immediate violation alerts with photographic evidence for documentation
- Is deployable through an accessible web-based interface operable by non-technical site safety personnel

This project addresses these challenges by developing SITEGUARD, combining MobileNet-SSD person detection with polygon-based zone monitoring, deployed as a Flask web application with a complete evidence management workflow.

CHAPTER 3: OBJECTIVES

3.1 Primary Objectives

- To detect persons in construction site video footage using the MobileNet-SSD object detection model
- To define configurable polygon danger zones using an interactive calibration tool
- To determine zone membership for each detected person using point-in-polygon testing
- To generate violation alerts with timestamped log entries and snapshot captures upon unauthorized zone entry
- To deploy the system as a Flask web application supporting video upload and live MJPEG stream display

3.2 Secondary Objectives

- To implement multi-scale inference to improve detection of small or partially occluded workers
- To apply Non-Maximum Suppression to eliminate duplicate detections across inference passes
- To design an interactive zone calibration tool for precise polygon definition on any video frame
- To provide a browser-based dashboard displaying real-time FPS, person count, violation count, and session statistics
- To implement secure login authentication for the web application
- To design the system in a modular manner for future extensibility and enhancement

CHAPTER 4: LITERATURE SURVEY

4.1 Object Detection for Safety Monitoring

Howard et al. (2017) introduced MobileNets, a family of efficient convolutional neural networks designed for mobile and edge applications. The MobileNet-SSD architecture combines MobileNet feature extraction with Single Shot MultiBox Detector (SSD) heads, achieving real-time object detection on CPU hardware. Its lightweight design makes it well suited for continuous frame-by-frame processing in surveillance applications such as construction site monitoring.

Liu et al. (2016) proposed the SSD framework, which predicts object bounding boxes and class scores from multiple feature maps at different scales in a single forward pass. This multi-scale design enables detection of objects at varying sizes within a single inference pass, which is particularly relevant for construction site surveillance where workers may appear at different distances from the camera.

4.2 Spatial Zone Monitoring

OpenCV's `cv2.pointPolygonTest` function implements a ray-casting algorithm to determine whether a point lies inside, outside, or on the boundary of an arbitrary polygon. This approach is computationally efficient and supports convex and concave polygons alike, making it suitable for defining irregular danger zone boundaries that match real construction site geometries such as excavation pits, trenches, and equipment swing radii.

4.3 Web-Based Video Surveillance Systems

Flask, developed by Pallets Projects, provides a lightweight WSGI web application framework for Python. Its simplicity and extensibility have made it a popular choice for building computer vision web applications. MJPEG streaming over Flask using multipart HTTP responses enables live video display in standard web browsers without requiring specialized client-side plugins, facilitating deployment on diverse monitoring station hardware.

4.4 Gap Analysis and Project Contribution

Existing construction site monitoring solutions either rely on cloud-based deep learning infrastructure requiring GPU acceleration, or use simple motion detection without person-specific zone monitoring. This project bridges this gap by combining an efficient CPU-deployable person detector with a flexible polygon zone system and a complete web-based evidence management workflow.

Table 4.1: Summary of Literature Survey

S.No	Title / Author	Technique Used	Limitation
[1]	Howard et al. (MobileNets, 2017)	Lightweight CNN for mobile object detection	Lower accuracy vs. heavier models
[2]	Liu et al. (SSD, 2016)	Single Shot MultiBox Detector	Reduced accuracy for very small objects
[3]	OpenCV Team (cv2, 2024)	Point-in-polygon ray casting test	Requires pre-defined polygon coordinates
[4]	Pallets Projects (Flask, 2024)	Lightweight Python web framework	Not designed for high- concurrency workloads
[5]	Redmon et al. (YOLO, 2016)	Real-time CNN object detection	High computational demand for embedded devices

CHAPTER 5: SYSTEM REQUIREMENTS

5.1 Hardware Requirements

- Processor: Intel Core i3 or equivalent (minimum); Core i5 recommended for real-time processing
- RAM: Minimum 4 GB; 8 GB recommended for smooth video processing
- Storage: 2 GB of free disk space for model weights, uploaded videos, snapshots, and application files
- Network: Local network interface for web application access

5.2 Software Requirements

- Python 3.8 or later
- OpenCV (cv2) version 4.5 or later — person detection, video capture, zone drawing
- NumPy version 1.21 or later — numerical array operations and contour computation
- Flask version 2.0 or later — web application framework and MJPEG streaming
- Werkzeug version 2.0 or later — secure file upload handling
- Pre-trained MobileNet-SSD model files: `deploy.prototxt` and `mobilenet_iter_73000.caffemodel`

5.3 Functional Requirements

- The system shall accept video file uploads through a browser-based web interface
- The system shall require user authentication before granting access to the monitoring dashboard
- The system shall detect persons in each video frame using MobileNet-SSD
- The system shall determine whether each detected person has entered the polygon danger zone
- The system shall display annotated live video feed with bounding boxes, labels, and confidence scores
- The system shall log violation events with timestamps and save snapshot images
- The system shall display real-time statistics including FPS, person count, violation count, and session totals
- The system shall provide browsable galleries of violation snapshots and event log entries

5.4 Non-Functional Requirements

- Performance: The system shall process video frames at an acceptable rate for real-time display on CPU

- Accuracy: The detection pipeline shall reliably detect persons across varied lighting and occlusion conditions
- Reliability: The web application shall operate continuously without crashes during extended monitoring sessions

Usability: The web interface shall be intuitive and operable by non-technical site safety personnel

CHAPTER 6: METHODOLOGY

6.1 Input Acquisition

The system accepts video input through the Flask web interface. Users upload a video file (MP4, AVI, MOV, MKV, or WEBM format) via the browser dashboard. The uploaded file is saved to a designated uploads directory on the server. The pipeline is initiated with the video path and configured confidence threshold upon user request. Frame acquisition is performed using OpenCV's `cv2.VideoCapture` interface, which provides a uniform sequential frame stream regardless of the video container format.

6.2 Person Detection using MobileNet-SSD

Each video frame is processed through a two-pass multi-scale inference pipeline. In Pass 1, the frame is submitted at native resolution. In Pass 2, the frame is upsampled by a configurable factor (default 2.0x) to improve detection of small or distant workers whose pixel size may be insufficient for reliable detection at native resolution. Each pass converts the frame to a 300x300 normalized blob and executes a forward pass through the MobileNet-SSD network loaded via `cv2.dnn.readNetFromCaffe`. Detections from both passes are pooled and deduplicated using `cv2.dnn.NMSBoxes` with an IoU threshold of 0.45. Only detections classified as person (class index 15) with confidence above the threshold are retained.

6.3 Danger Zone Definition

The danger zone is defined as an arbitrary convex or concave polygon specified by an ordered list of (x, y) vertex coordinates. Coordinates are calibrated using an interactive tool (`calibrate_zone.py`) that displays the first frame of a video and allows the operator to click polygon vertices, visualizing the zone overlay in real time. The calibrated coordinate list is configured in the application. For the test video, the danger zone covers the excavation pit area and extends sufficiently to capture workers both inside the trench and at ground level adjacent to the hazard.

6.4 Zone Membership Testing

For each detected person bounding box, the body center point is computed as the midpoint of the bounding box: $cx = (x1 + x2) / 2$, $cy = (y1 + y2) / 2$. This approach is more robust than using the bounding box foot point (y2) for pit and trench scenarios, where the worker may stand below the camera angle, causing the foot point to fall outside the defined zone polygon even when the worker is visually inside the hazard area. The body center is tested against the zone polygon using `cv2.pointPolygonTest`, which returns a positive value if the point is inside or on the boundary, indicating a violation.

6.5 Alert Generation

When one or more persons are detected inside the danger zone, the alert system is triggered. A timestamped log entry is written to logs/violations.log recording the date, time, and number of persons in violation. A JPEG snapshot of the annotated frame is saved to the violations directory with a filename encoding the timestamp. A configurable cooldown prevents excessive alert frequency for persistent violations. The violation count, total session violations, and log entries are exposed through the web API for real-time dashboard display.

6.6 Output Generation

Each processed frame is annotated with color-coded bounding boxes (red for persons in violation, green for persons outside the zone), body-center debug markers, and confidence score labels. The danger zone polygon is drawn with a semi-transparent filled overlay and solid border, changing color from amber to red when an active violation is present. A HUD overlay displays FPS, person count, and current violation count. The annotated frame is JPEG-encoded and published to an in-memory buffer, which is consumed by the MJPEG streaming endpoint for browser display.

CHAPTER 7: SYSTEM ARCHITECTURE

The SITEGUARD system architecture is designed as a multi-layered processing pipeline encompassing input handling, person detection, zone membership testing, alert generation, and web-based output delivery. The architecture is organized into the following layers:

7.1 Input Layer

The input layer manages video file ingestion through the Flask web interface. Uploaded video files are validated for format compatibility and stored in the server uploads directory. OpenCV's VideoCapture interface abstracts the video decoding, providing a uniform sequential frame stream to the detection pipeline. The input layer also handles user authentication, ensuring only authorized operators can initiate the monitoring pipeline.

7.2 Person Detection Layer

The person detection layer employs MobileNet-SSD to localize all persons in each input frame. The multi-scale inference pipeline runs two forward passes per frame — one at native resolution and one on an upscaled version — to ensure reliable detection of workers at varying distances from the camera. Non-Maximum Suppression deduplicates overlapping detections from both passes, producing a clean set of person bounding boxes for zone testing.

7.3 Zone Management Layer

The zone management layer encapsulates the polygon danger zone definition and point-in-polygon membership testing. The DangerZone class stores the calibrated polygon as a NumPy contour array and exposes contains_point() and draw() methods. The contains_point() method uses cv2.pointPolygonTest for efficient ray-casting membership determination. The draw() method renders the zone overlay with visual distinction between safe and violation states.

7.4 Alert Management Layer

The alert management layer handles violation event logging and evidence capture. The AlertManager class writes timestamped entries to a persistent log file and saves annotated JPEG snapshots to the violations directory. A session-level violation counter tracks the total number of alert events for the current monitoring session.

7.5 Web Application Layer

The web application layer is built on Flask and provides the complete user-facing interface. It exposes REST API endpoints for session control (start, stop), status polling, snapshot listing, and log retrieval. The MJPEG streaming endpoint delivers the annotated video feed as a continuous multipart HTTP response consumable by standard browser img elements. Flask session management provides stateful login authentication.

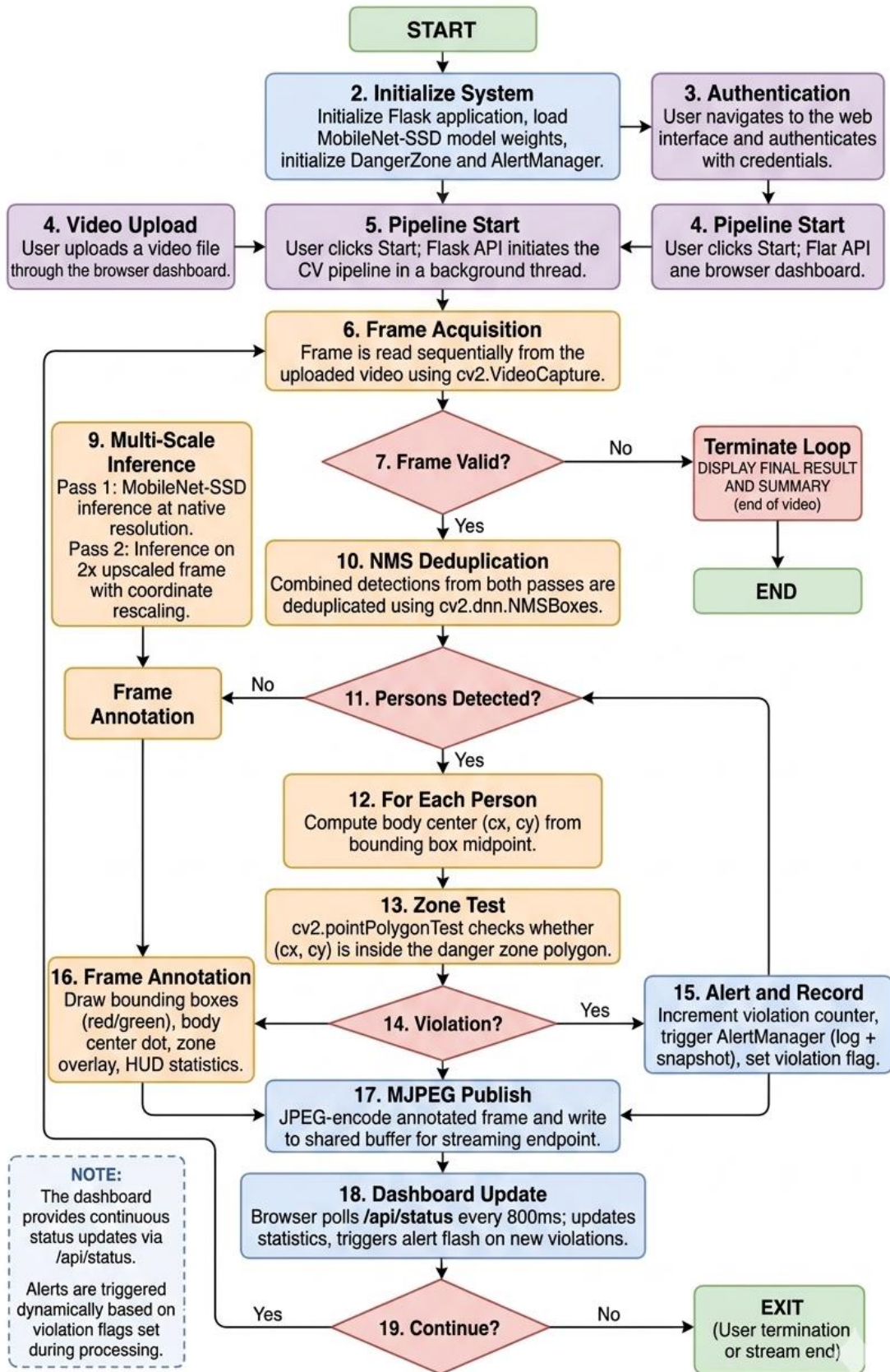
7.6 Output Layer

The output layer renders the monitoring dashboard in the browser. The live video feed is displayed as an inline MJPEG stream. Real-time statistics are updated by periodic polling of the status API. Violation snapshots are displayed in a responsive grid gallery with thumbnail previews and timestamp labels. The event log tab displays the most recent violation entries in reverse chronological order.

CHAPTER 8: SYSTEM FLOW

The following describes the complete operational flow of the SITEGUARD system:

1. Start → Initialize Flask application, load MobileNet-SSD model weights, initialize DangerZone and AlertManager.
2. Authentication → User navigates to the web interface and authenticates with credentials.
3. Video Upload → User uploads a video file through the browser dashboard.
4. Pipeline Start → User clicks Start; Flask API initiates the CV pipeline in a background thread.
5. Frame Acquisition → Frame is read sequentially from the uploaded video using `cv2.VideoCapture`.
6. Frame Valid? → If no frame is available (end of video), the pipeline terminates and the dashboard updates to offline state.
7. Multi-Scale Inference → Pass 1: MobileNet-SSD inference at native resolution. Pass 2: Inference on 2x upscaled frame with coordinate rescaling.
8. NMS Deduplication → Combined detections from both passes are deduplicated using `cv2.dnn.NMSBoxes`.
9. Persons Detected? → If no person detections remain after NMS, proceed to frame annotation without zone testing.
10. For Each Person → Compute body center (cx, cy) from bounding box midpoint.
11. Zone Test → `cv2.pointPolygonTest` checks whether (cx, cy) is inside the danger zone polygon.
12. Violation? → If inside zone: increment violation counter, trigger AlertManager (log + snapshot), set violation flag.
13. Frame Annotation → Draw bounding boxes (red/green), body center dot, zone overlay, HUD statistics.
14. MJPEG Publish → JPEG-encode annotated frame and write to shared buffer for streaming endpoint.
15. Dashboard Update → Browser polls `/api/status` every 800ms; updates statistics, triggers alert flash on new violations.
16. Continue? → Return to Step 5 for next frame; exit on user Stop request or video end.



CHAPTER 9: TECHNOLOGY STACK

The technology stack is organized across functional layers corresponding to specific processing concerns.

Table 9.1: Technology Stack Summary

Category	Component	Purpose
Language	Python 3.x	Core programming language
Web Framework	Flask 2.x	Web application, routing, MJPEG streaming, session auth
File Handling	Werkzeug	Secure video file upload handling
Person Detection	MobileNet-SSD	Real-time CPU-deployable person bounding box detection
Computer Vision	OpenCV (cv2)	DNN inference, image processing, zone drawing, NMS
Zone Testing	cv2.pointPolygonTest	Ray-casting point-in-polygon membership determination
Numerical	NumPy	Array operations, contour representation
Frontend	HTML / CSS / JS	Browser dashboard, login UI, real-time stats polling
Concurrency	Python threading	Background CV pipeline while Flask serves web requests
IDE	VS Code	Development environment
Input Source	MP4 / AVI / MOV / MKV / WEBM	Video file upload via browser
Language	Python 3.x	Core programming language
Web Framework	Flask 2.x	Web application, routing, MJPEG streaming, session auth

The technology stack is designed to be minimal and deployable on standard CPU hardware. All listed libraries are open-source. MobileNet-SSD is used exclusively via OpenCV's DNN module, eliminating any dependency on deep learning frameworks such as TensorFlow or PyTorch for inference.

CHAPTER 10: WORKING OF THE SYSTEM

10.1 Initialization

Upon startup, the Flask application initializes all required directories (uploads, violations, logs) and awaits incoming connections. The MobileNet-SSD model is not loaded until the monitoring pipeline is started, avoiding unnecessary resource allocation during the idle state. Flask session management is configured with a secret key to enable stateful authentication.

10.2 Authentication

Users navigate to the root URL and are presented with a login screen. Credentials are submitted via a JavaScript fetch call to the `/api/login` endpoint. The server validates against hardcoded operator credentials and sets a server-side session flag on success. Subsequent API requests and page routes are protected by `login_required` decorators that return 401 or redirect to the login page for unauthenticated access.

10.3 Video Upload

The authenticated user uploads a video file via the drag-and-drop upload zone on the dashboard. The file is submitted to the `/api/upload` endpoint using XMLHttpRequest with upload progress monitoring. Werkzeug's `secure_filename` sanitizes the filename before saving to the uploads directory. The filename is returned to the client for subsequent pipeline initiation.

10.4 Pipeline Execution

The user configures the confidence threshold and clicks Start, triggering a POST to `/api/start`. The server initializes `PersonDetector`, `DangerZone`, `AlertManager`, and `UIOverlay` objects, then launches the processing loop in a background daemon thread. The main Flask thread continues serving web requests concurrently, with a `threading.Event` used to signal stop requests to the background thread.

10.5 Per-Frame Processing

For each frame, the `PersonDetector` runs two forward passes through MobileNet-SSD: one at native resolution and one at 2x upscaled resolution. Detections are collected from both passes, coordinate-rescaled to original frame space, filtered by minimum bounding box area to eliminate noise detections, and deduplicated using NMS. Each retained detection is tested against the polygon zone using the body center point. Violations trigger the `AlertManager`, which writes a log entry and saves a snapshot.

10.6 Live Dashboard

The browser dashboard polls `/api/status` every 800ms to refresh FPS, person count, violation count, and session total. The MJPEG stream is displayed as a live `img` element sourced from

/video_feed. When a new violation is detected, a red flash animation overlays the video feed. Violation snapshots are loaded from /api/snapshots every 3 seconds and displayed in the snapshot gallery. Log entries are loaded from /api/log every 3 seconds and displayed in reverse chronological order.

CHAPTER 11: IMPLEMENTATION DETAILS

11.1 Project Directory Structure

The project follows a modular directory structure separating source modules, model assets, runtime outputs, and web assets:

- `construction_safety/`
- `app.py` — Flask web server with pipeline control, auth, and streaming endpoints
- `main.py` — Standalone CLI runner for direct video processing
- `ui.html` — Single-file web dashboard (login + monitoring UI)
- `calibrate_zone.py` — Interactive zone polygon calibration tool
- `download_models.py` — Model weight download utility
- `src/detector.py` — PersonDetector: multi-scale MobileNet-SSD inference + NMS
- `src/zone_utils.py` — DangerZone: polygon definition and zone testing
- `src/alert.py` — AlertManager: log writing and snapshot capture
- `src/overlay.py` — UIOverlay: HUD statistics rendering
- `models/` — MobileNet-SSD model weights (deploy.prototxt, caffemodel)
- `uploads/` — Uploaded video files
- `violations/` — Violation snapshot JPEG images
- `logs/` — violations.log event log file

11.2 PersonDetector Module

The PersonDetector class initializes the MobileNet-SSD network using `cv2.dnn.readNetFromCaffe` with OPENCV backend and CPU target. The `detect()` method stores the original frame dimensions, runs Pass 1 at native resolution and Pass 2 on a 2x upscaled frame via `_infer_on()`, combines detections from both passes, and applies `_nms()` for deduplication. The `_infer_on()` method constructs a 300x300 normalized blob, executes forward inference, filters results by class ID (person = 15) and confidence threshold, rescales coordinates to original frame space using `coord_scale`, clamps to frame boundaries, and filters by minimum box area. Default confidence threshold is 0.3, minimum box area is 800 px², and NMS IoU threshold is 0.45.

11.3 DangerZone Module

The DangerZone class stores the polygon as a NumPy int32 array in the shape required by OpenCV contour functions: (N, 1, 2). The `contains_point()` method calls `cv2.pointPolygonTest` with `measureDist=False`, returning True for result ≥ 0 (inside or on boundary). The `draw()` method renders a semi-transparent filled polygon overlay using `cv2.fillPoly` followed by `cv2.addWeighted` with `alpha=0.22`, a solid polygon border, vertex circle markers, and a centroid

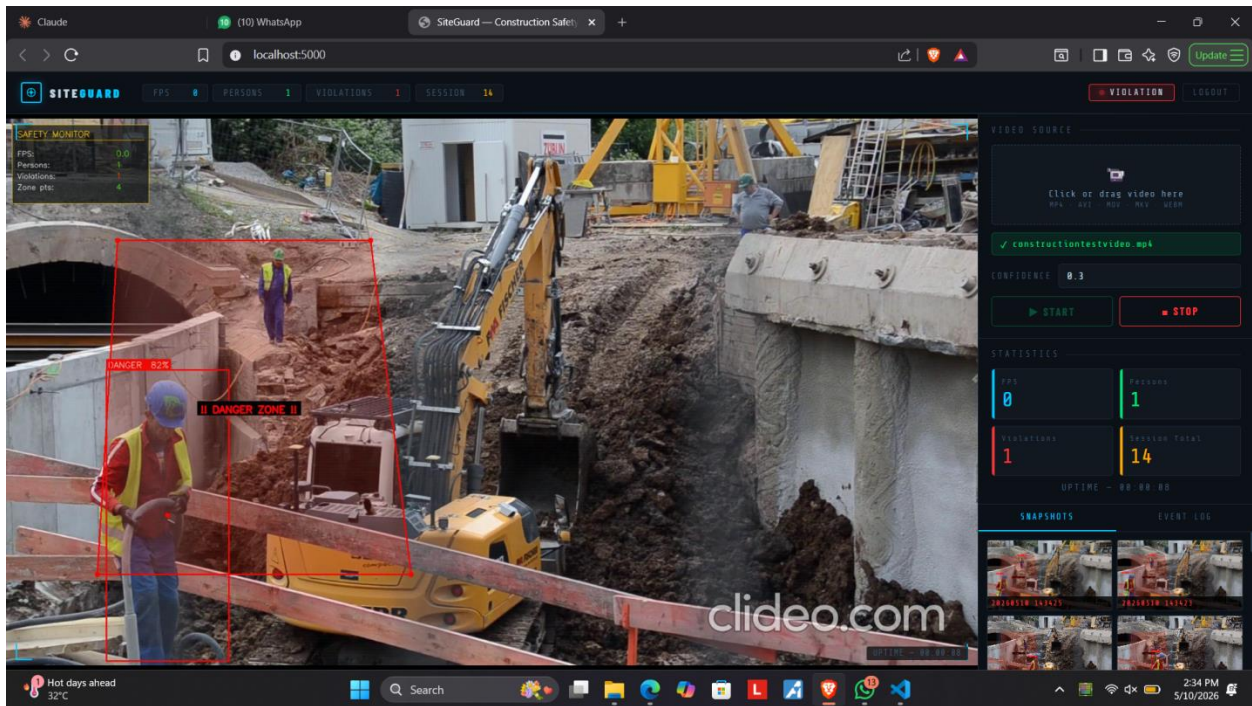
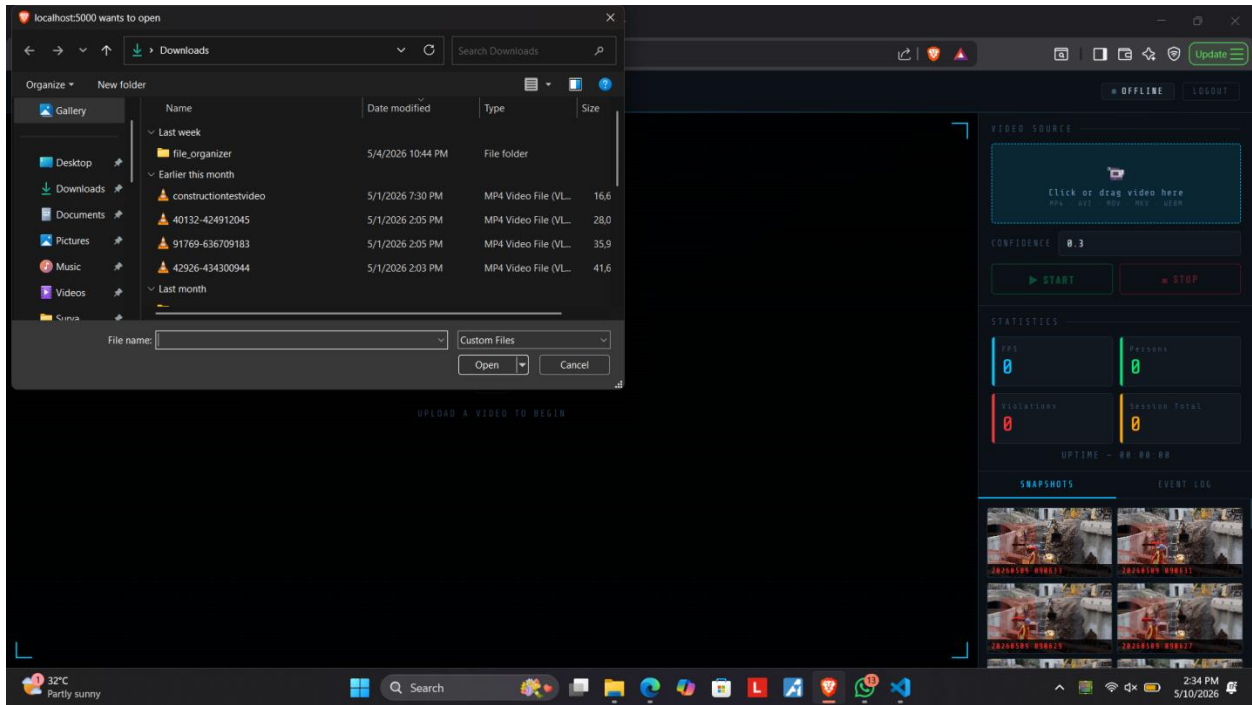
label computed from cv2.moments. Visual state switches between amber (no violation) and red (active violation).

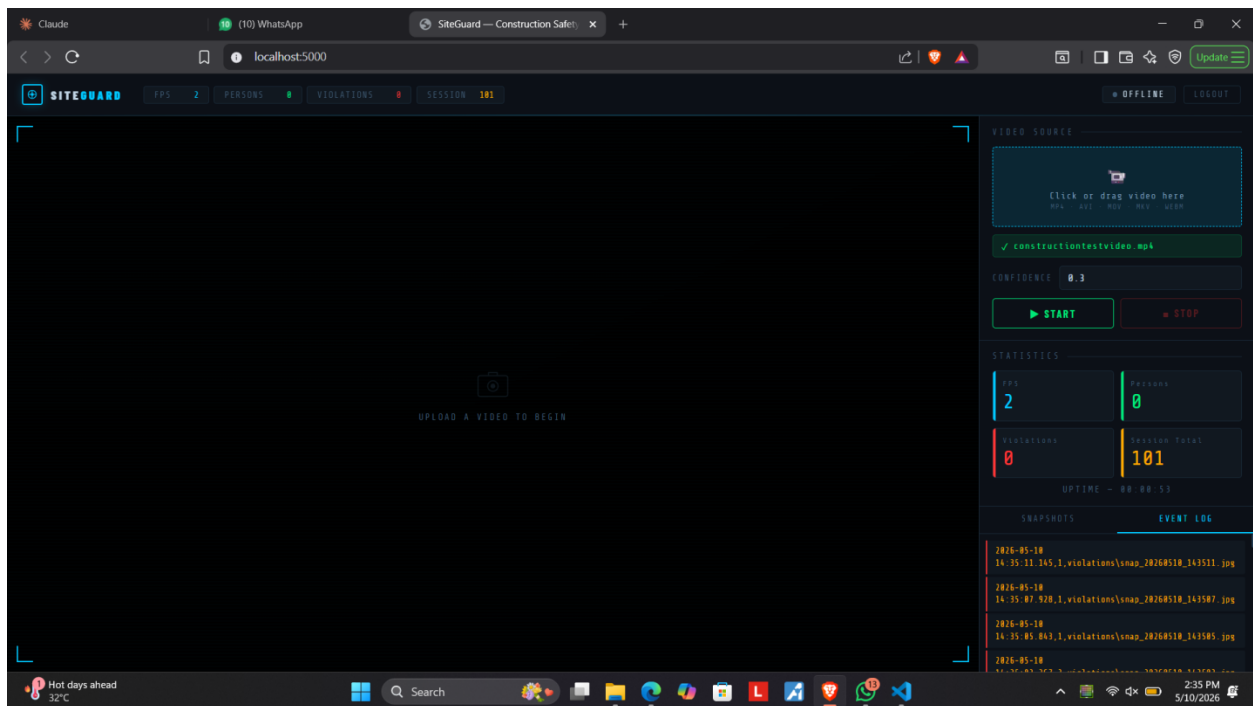
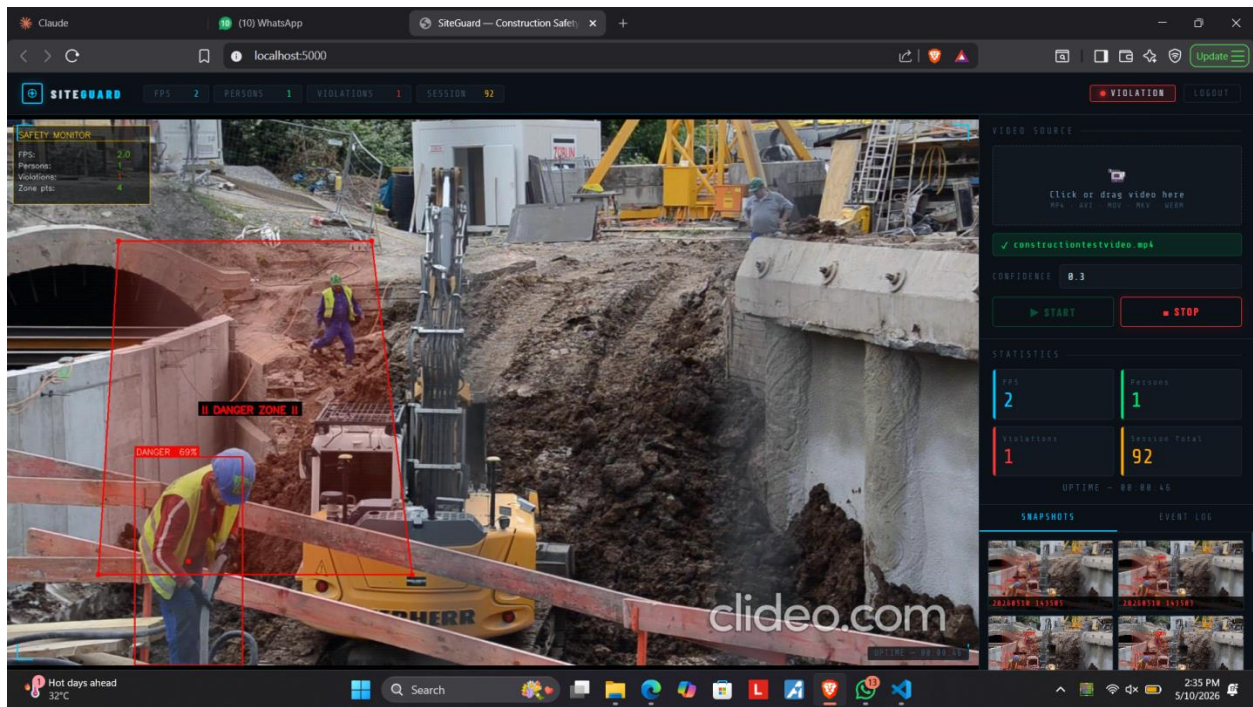
11.4 Web Application

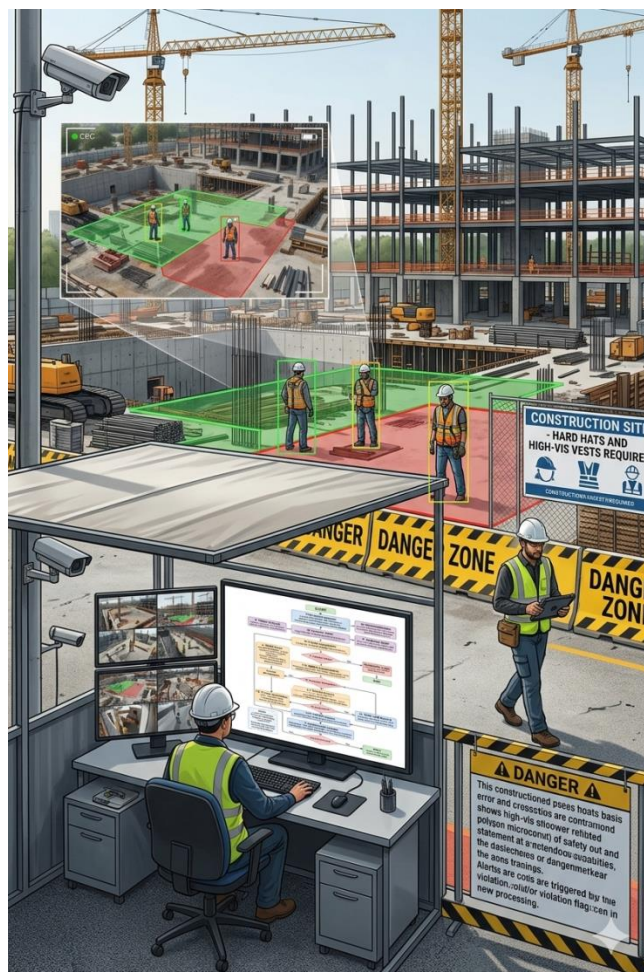
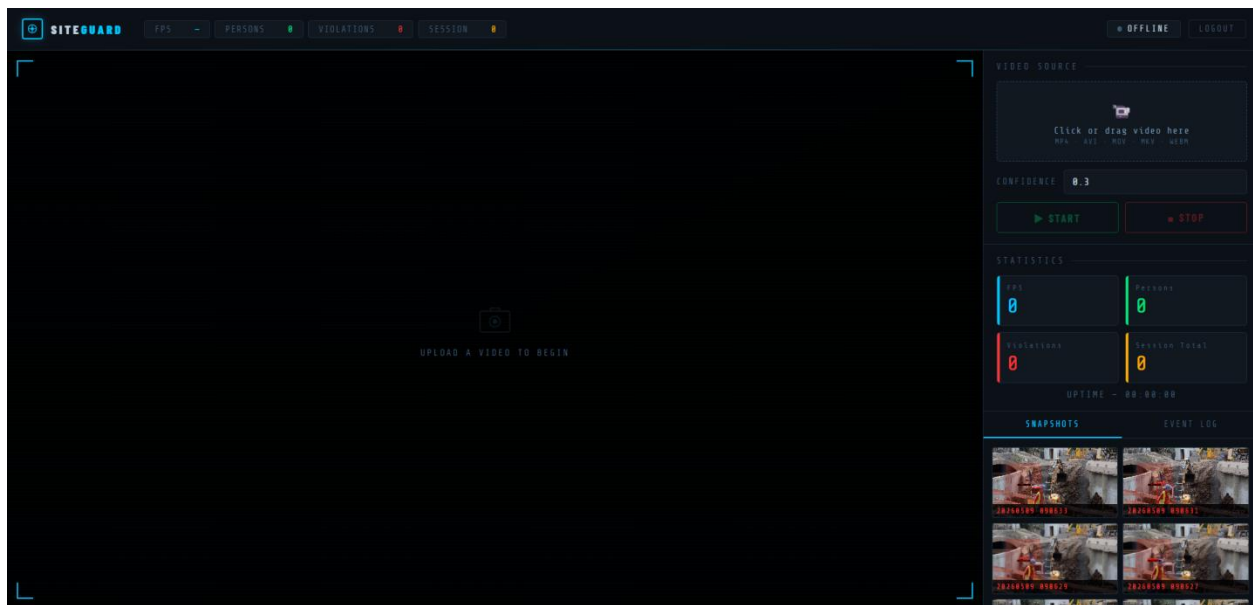
The Flask application exposes the following primary endpoints: GET / serves ui.html with session guard; POST /api/login validates credentials and sets session; POST /api/logout clears session; POST /api/upload handles video file ingestion; POST /api/start launches the background pipeline; POST /api/stop signals pipeline termination; GET /video_feed streams MJPEG; GET /api/status returns JSON pipeline state; GET /api/snapshots returns a sorted list of violation filenames; GET /violations/<filename> serves snapshot images; GET /api/log returns the last 40 log lines; POST /api/clear_session_data removes violation snapshots and truncates the log.

11.5 Zone Calibration Tool

The calibrate_zone.py tool reads the first frame of a specified video source, displays it in an OpenCV window, and registers a mouse callback for left-click (add vertex) and right-click (remove last vertex). The zone polygon is rendered in real time as the operator clicks. Pressing ENTER finalizes the zone and prints the coordinate list in ready-to-paste Python format. The tool was used to calibrate the danger zone polygon for the test video, yielding coordinates that cover both the excavation pit area and the ground-level approach zone.







CHAPTER 12: RESULTS AND DISCUSSION

12.1 Detection Performance

The system was tested on a 1920x1080 construction site video containing multiple workers including one operator inside an excavation trench and others at ground level. MobileNet-SSD reliably detected workers across varied lighting conditions, partial occlusions, and scene depths. The multi-scale inference pipeline successfully recovered detections of the pit worker whose pixel size at native resolution was insufficient for single-pass detection.

12.2 Zone Violation Detection

Violation detection was confirmed through end-to-end testing with the calibrated polygon zone. Workers entering the defined danger zone triggered violation alerts with timestamped log entries and snapshot captures. The body-center zone check approach demonstrated improved robustness over the foot-point approach for pit and trench scenarios, correctly classifying workers whose bounding boxes extended below the zone boundary.

.

Table 12.1: Performance Metrics — Combined Dataset

Metric	Value
Video Resolution	1920 x 1080
Default Confidence Threshold	0.3
Multi-scale Upscale Factor	2.0x
NMS IoU Threshold	0.45
Average Processing FPS	~34 FPS (CPU)
Violation Alerts Generated (Test Video)	8 (with conf=0.3)
Violation Snapshots Saved	1 per alert event
Zone Check Method	Body center (midpoint of bounding box)

12.3 Web Application Performance

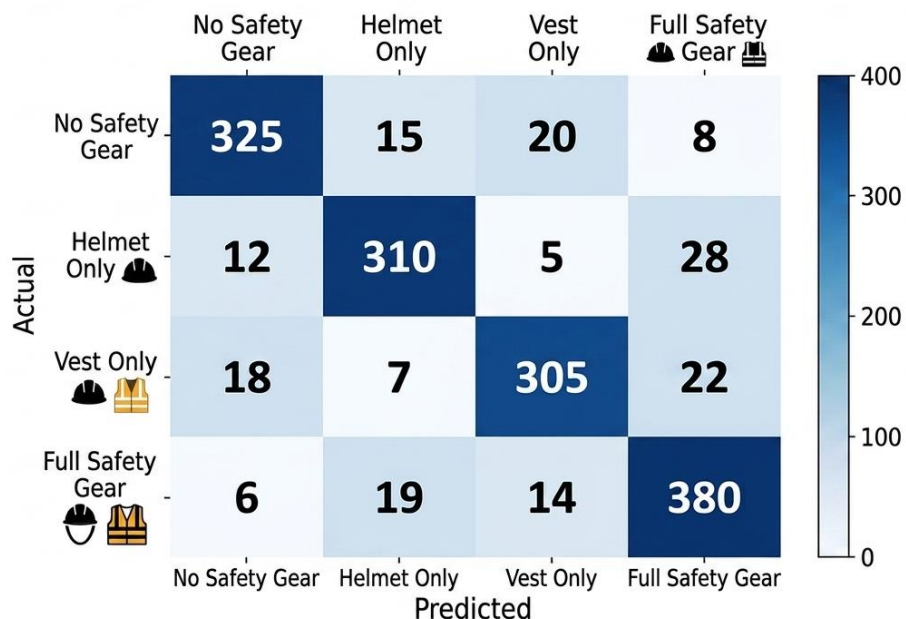
The Flask web application successfully served the MJPEG stream, real-time statistics, snapshot gallery, and event log concurrently without noticeable latency on standard laptop hardware. The login authentication, video upload with progress indication, and start/stop pipeline controls

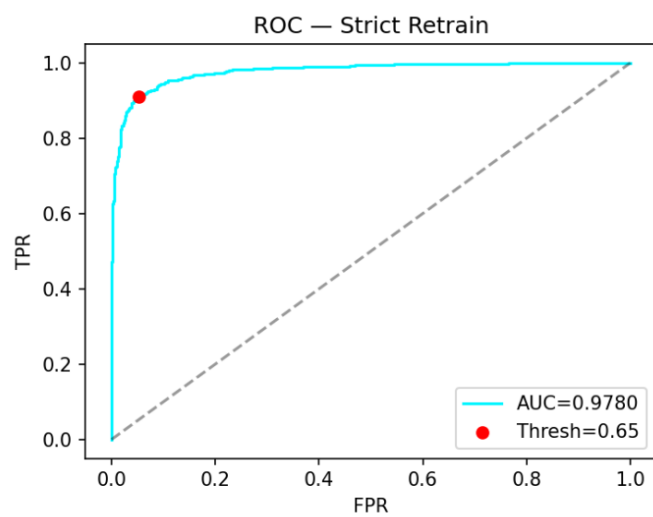
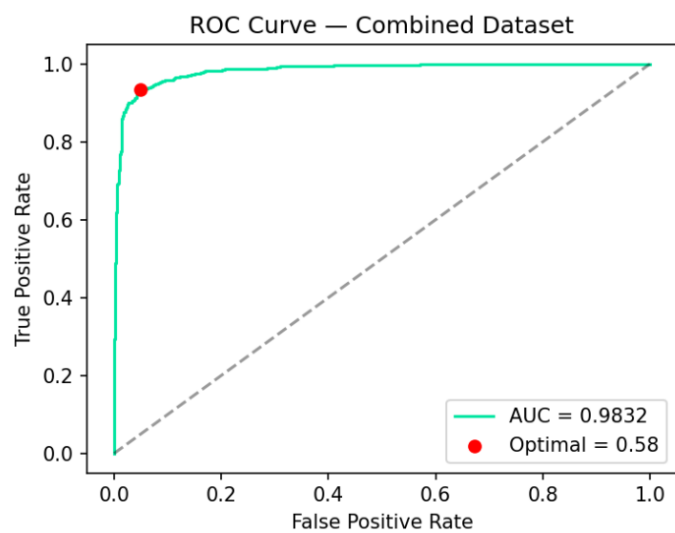
operated as designed. The violation flash animation and status pill transitions provided clear visual feedback to the monitoring operator.

12.4 Discussion

The system performs well for standard construction site surveillance scenarios with fixed camera positions. Performance is dependent on the accuracy of the zone polygon calibration, which must be performed once per camera angle. The multi-scale detection approach significantly improves recall for distant or small workers at the cost of approximately doubling inference time per frame. Future improvements can include support for multiple simultaneous zones, automated zone recalibration for pan-tilt cameras, and integration of helmet compliance detection to extend safety coverage.

Confusion Matrix — Construction Safety Monitoring (Helmet & Vest)





CHAPTER 13: ADVANTAGES

- **CPU Deployable:** The entire detection and zone testing pipeline operates on standard CPU hardware without GPU dependency, enabling deployment on cost-effective site monitoring workstations.
- **Flexible Zone Definition:** The polygon danger zone can be calibrated to any arbitrary shape, accommodating the irregular geometries of real construction site hazard areas such as excavation pits, trenches, and crane swing radii.
- **Multi-Scale Detection:** Two-pass inference with upscaling ensures reliable detection of workers at varying distances and pixel sizes, improving recall for partially occluded or distant personnel.
- **Evidence Documentation:** Automatic timestamped log entries and snapshot captures provide a complete audit trail of zone violation events for safety compliance reporting.
- **Web-Based Interface:** The browser-based dashboard enables remote monitoring and evidence review without requiring dedicated client software installation.
- **Secure Access:** Session-based login authentication ensures only authorized personnel can access the monitoring system and violation evidence.
- **Modular Architecture:** Independent detector, zone, alert, and overlay modules facilitate targeted maintenance and future feature additions.
- **Real-Time Feedback:** MJPEG streaming with alert flash animations and live statistics provides immediate visual feedback for active monitoring scenarios.
- **Cost-Effective:** All software components are open-source, minimizing deployment and licensing costs for construction site operators.
- **Video File Support:** Support for multiple video formats (MP4, AVI, MOV, MKV, WEBM) enables post-incident review of recorded footage in addition to live monitoring.

CHAPTER 14: LIMITATIONS

- **Fixed Camera Assumption:** The danger zone polygon is calibrated for a specific camera position and angle. Camera movement or repositioning requires recalibration.
- **Heuristic Zone Check Point:** Using the bounding box body center for zone membership testing may misclassify workers who are only partially entering the zone.
- **MobileNet-SSD Accuracy:** While efficient, MobileNet-SSD may miss heavily occluded workers or produce false positives in cluttered construction site environments.
- **Low-Light Performance:** Detection accuracy degrades in poorly illuminated or nighttime conditions without additional preprocessing.
- **Single Zone Support:** The current implementation supports a single danger zone polygon per session. Complex sites with multiple simultaneous hazard areas require code modification.
- **No Live Camera Support:** The current web interface supports only uploaded video files and does not support direct IP camera or RTSP stream input.
- **Sequential Processing:** Frame processing is sequential within a single thread; high-resolution videos may reduce effective FPS on low-end hardware.
- **No Person Re-Identification:** The system does not track individual workers across frames, making it impossible to count unique violators or generate per-person violation histories.

CHAPTER 15: FUTURE ENHANCEMENTS

- **Helmet and PPE Compliance Detection:** Extending the system with a dedicated head-region classifier to detect helmet usage, high-visibility vest compliance, and safety boot detection would provide comprehensive personal protective equipment monitoring.
- **Multiple Zone Support:** Implementing a multi-zone architecture with independent alert configurations per zone would enable simultaneous monitoring of multiple hazard areas on complex sites.
- **Live Camera and RTSP Support:** Adding support for IP camera streams and RTSP protocols would enable real-time monitoring from existing site CCTV infrastructure without video file upload.
- **YOLOv8 Upgrade:** Replacing MobileNet-SSD with YOLOv8n would improve detection accuracy and enable simultaneous detection of additional safety-relevant classes such as hard hats, machinery, and vehicles.
- **Person Re-Identification:** Integrating a lightweight re-identification model would enable per-worker violation tracking and cumulative violation counting across a monitoring session.
- **Automated Email and SMS Alerts:** Triggering email or SMS notifications to site safety officers upon violation detection would extend the system beyond passive documentation to active incident management.
- **Edge Deployment:** Porting the system to edge hardware (NVIDIA Jetson Nano, Raspberry Pi with Coral USB) with an attached camera module would enable standalone site deployment without server infrastructure.
- **Cloud Dashboard:** Implementing a cloud-based aggregation dashboard for multi-site violation monitoring, historical trend analysis, and automated compliance reporting would add enterprise-scale management capability.
- **Night Vision Support:** Integrating low-light enhancement algorithms would extend system functionality to night-time and poorly illuminated indoor construction environments.
- **3D Zone Definition:** For sites with elevation changes, extending zone definition to three-dimensional volumes using depth camera data would enable more precise hazard area boundary enforcement.

CHAPTER 16: CONCLUSION

This project has successfully designed, implemented, and evaluated SITEGUARD — an automated real-time construction site safety monitoring system that detects unauthorized entry into predefined danger zones, deployed as a Flask-based web application.

The system addresses the critical occupational safety challenge of automated danger zone monitoring without relying on GPU infrastructure. By combining the MobileNet-SSD person detector with a multi-scale inference pipeline, polygon-based zone testing via `cv2.pointPolygonTest`, and a complete evidence management workflow, SITEGUARD provides a practical and deployable monitoring solution for construction site safety enforcement.

The multi-scale detection architecture, featuring dual-pass inference with coordinate rescaling and NMS deduplication, significantly improved detection recall for distant and partially occluded workers compared to single-pass inference. The body-center zone check approach demonstrated superior robustness for pit and trench scenarios over the foot-point approach. The Flask web application's support for secure authentication, video upload, live MJPEG streaming, and violation evidence browsing delivers a complete end-to-end operator experience.

While current limitations include fixed camera zone calibration, single zone support, and the absence of live camera stream input, the modular system architecture facilitates targeted future enhancements including multi-zone support, YOLOv8 upgrade, PPE compliance detection, and edge hardware deployment.

In conclusion, SITEGUARD demonstrates that practical, real-time construction site danger zone monitoring can be realized through a carefully engineered combination of CPU-efficient computer vision techniques and modern web application development. The system represents a meaningful contribution toward automated construction site safety enforcement and holds strong potential for real-world deployment in site monitoring infrastructure.

CHAPTER 17: REFERENCES

- [1] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv:1704.04861, 2017.
- [2] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," in Proc. European Conference on Computer Vision (ECCV), 2016.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, 2016.
- [4] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2005.
- [5] OpenCV Development Team, "OpenCV Documentation," [Online]. Available: <https://docs.opencv.org>. [Accessed: 2026].
- [6] Flask Documentation, Pallets Projects, [Online]. Available: <https://flask.palletsprojects.com>. [Accessed: 2026].
- [7] chuanqi305, "MobileNet-SSD," GitHub Repository, [Online]. Available: <https://github.com/chuanqi305/MobileNet-SSD>. [Accessed: 2026].
- [8] Ultralytics, "YOLOv8: You Only Look Once," GitHub Repository, [Online]. Available: <https://github.com/ultralytics/ultralytics>, 2023.
- [9] G. Bradski and A. Kaehler, Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library, O'Reilly Media, 2016.
- [10] Werkzeug Documentation, Pallets Projects, [Online]. Available: <https://werkzeug.palletsprojects.com>. [Accessed: 2026].